

Documentación de Replicación y Consistencia (PostgreSQL)

Sistema C5 de Alertas Ciudadanas

Objetivo

El componente de base de datos del Sistema C5 de Alertas Ciudadanas fue diseñado para garantizar una alta disponibilidad, escalabilidad y rapidez en el procesamiento de alertas provenientes de dispositivos ESP32. Para lograrlo, se implementó una arquitectura de replicación Maestro–Réplica utilizando PostgreSQL desplegado mediante contenedores Docker. Esta estrategia permite distribuir la carga de trabajo entre los nodos de escritura y lectura, optimizando el rendimiento general del sistema y asegurando una respuesta eficiente ante eventos de emergencia.

1. Arquitectura Maestro–Réplica

La infraestructura de base de datos está configurada mediante dos contenedores PostgreSQL definidos dentro del archivo docker-compose.yaml:

Nodo Maestro (postgres-master)

El nodo maestro es el encargado de recibir todas las operaciones de escritura generadas por el sistema. Su función principal consiste en almacenar las alertas ciudadanas procesadas por los microservicios de recepción, clasificación y persistencia.

La configuración utiliza la imagen Bitnami PostgreSQL y se habilita el modo maestro mediante la variable:

POSTGRESQL_REPLICATION_MODE=master

Además, se configura un usuario de replicación dedicado que permite transferir los cambios realizados en la base de datos hacia los nodos secundarios

Las alertas generadas por los dispositivos ESP32 son procesadas por los microservicios y posteriormente almacenadas exclusivamente en este nodo para evitar conflictos y garantizar la integridad de los datos.

Nodo Réplica (postgres-replica)

El nodo réplica mantiene una copia sincronizada de la información almacenada en el maestro. Su principal responsabilidad es atender las consultas realizadas por los operadores del sistema sin afectar el rendimiento de las operaciones de escritura.

La réplica se configura mediante:

POSTGRESQL_REPLICATION_MODE=slave

Y establece conexión directa con el maestro utilizando:

POSTGRESQL_MASTER_HOST=postgres-master

De esta forma, todos los cambios realizados en el maestro son replicados automáticamente hacia el nodo secundario.

Beneficios de la arquitectura

La separación entre lecturas y escrituras proporciona las siguientes ventajas:

- Mayor rendimiento durante la recepción masiva de alertas.
- Reducción de la carga sobre el nodo principal.
- Escalabilidad horizontal para consultas históricas.

- Mejor disponibilidad del sistema.
- Menor tiempo de respuesta para los operadores del C5.

En consecuencia, el nodo maestro se especializa en procesar escrituras mientras que la réplica se dedica exclusivamente a las consultas de información histórica.

2. Enrutamiento de Lecturas (Lecturas en Réplica)

El microservicio Historial de Incidentes implementa una estrategia explícita de separación entre lectura y escritura utilizando dos conexiones independientes hacia PostgreSQL.

Durante la inicialización del servicio se crean dos motores de conexión:

- DB_MASTER = postgresql://c5_user@postgres-master:5432/c5_db
- DB_REPLICA = postgresql://c5_user@postgres-replica:5432/c5_db

Posteriormente se generan dos sesiones independientes:

- SessionMaster = sessionmaker(bind=engine_master)
- SessionReplica = sessionmaker(bind=engine_replica)

Escrituras dirigidas al Maestro

El proceso encargado de almacenar nuevas alertas utiliza exclusivamente SessionMaster.

Cada vez que una alerta es recibida desde Redis, el microservicio crea un registro AlertLog y ejecuta:

```
session = SessionMaster()  
session.add(db_log)  
session.commit()
```

Esto garantiza que toda modificación de datos ocurra únicamente en el nodo maestro.

Lecturas dirigidas a la Réplica

El endpoint REST encargado de consultar el historial utiliza exclusivamente SessionReplica.

En el método:

GET /api/history

se establece la sesión:

```
session = SessionReplica()
```

Todas las búsquedas, filtros y consultas son ejecutadas sobre la réplica.

Además, la respuesta de la API devuelve explícitamente:

```
"database_source": "replica_node"
```

lo que permite verificar que la información proviene del nodo secundario.

Ventajas del enrutamiento de lecturas

Esta estrategia ofrece varios beneficios:

- Evita que las consultas históricas compitan con las escrituras.
- Reduce la carga del nodo maestro.
- Permite atender múltiples operadores simultáneamente.
- Incrementa la escalabilidad del sistema.

- Mantiene un alto rendimiento durante situaciones de emergencia.

Por lo tanto, el historial de incidentes puede ser consultado constantemente sin afectar el procesamiento de nuevas alertas.

3. Modelo de Consistencia (Justificación Crítica)

Modelo seleccionado: **Consistencia Eventual**

La arquitectura implementada utiliza un modelo de consistencia eventual.

Este modelo se deriva de la utilización de un esquema Maestro–Réplica donde las escrituras se realizan en el nodo principal y posteriormente son propagadas hacia la réplica. Debido a que la sincronización no ocurre de manera instantánea, puede existir un pequeño intervalo de tiempo durante el cual una alerta ya está almacenada en el maestro pero aún no es visible en la réplica.

Por esta razón no se considera una consistencia fuerte, ya que las lecturas realizadas inmediatamente después de una escritura podrían no reflejar el estado más reciente de la base de datos.

Justificación para un Sistema C5

La elección de consistencia eventual resulta adecuada para un sistema de gestión de emergencias debido a que la prioridad principal es garantizar la disponibilidad y velocidad de recepción de alertas.

Cuando un ciudadano genera una alerta desde un dispositivo ESP32, el sistema debe procesarla y almacenarla lo más rápido posible. Si se utilizara consistencia fuerte con sincronización estricta entre nodos, cada escritura tendría que esperar la confirmación de todas las réplicas antes de completarse, aumentando la latencia y reduciendo el rendimiento.

En contraste, la consistencia eventual permite que el nodo maestro confirme inmediatamente la escritura y continúe procesando nuevas alertas mientras la réplica se actualiza en segundo plano.

En el contexto operativo del C5, un retraso de algunos milisegundos o segundos en la visualización del historial no representa un problema crítico, ya que las notificaciones y acciones de respuesta inmediata son gestionadas por otros componentes del sistema, como Redis, MQTT y los microservicios de procesamiento en tiempo real.

Beneficios de la Consistencia Eventual

- Mayor velocidad de escritura.
- Menor latencia durante emergencias.
- Mejor disponibilidad del sistema.
- Mayor capacidad para manejar grandes volúmenes de alertas.
- Escalabilidad para futuras expansiones.

Conclusión

La implementación de PostgreSQL mediante una arquitectura Maestro–Réplica permite separar eficientemente las operaciones de escritura y lectura dentro del Sistema C5. Las alertas son almacenadas en el nodo maestro mientras que las consultas históricas son atendidas por la réplica, optimizando el rendimiento global. Debido a que la replicación ocurre de forma diferida, el sistema adopta un modelo de consistencia eventual, el cual resulta apropiado para entornos de emergencia donde la disponibilidad, velocidad de procesamiento y capacidad de respuesta son más importantes que la sincronización instantánea de todas las copias de datos.